

Introduction to OpenModelica for Process Engineers



BY

VIRAJ DESAI, PROCESS ENGINEER

EMAIL: VIRAJ DESAI

LEVEL: BEGINNER

VERSION: 0.00

4-SEP-26

Preface

Process Simulation in OpenModelica is designed as a foundational, hands-on guide for engineers, researchers, and students who wish to explore the world of dynamic process simulation using a free and open-source tool.

This manual provides a preliminary yet structured overview of the OpenModelica environment. Rather than diving directly into complex industrial case studies, we focus on building a solid foundation: understanding what OpenModelica is, installing it correctly, navigating its user interface, mastering the basics of the Modelica language, and confidently exporting simulation data for further analysis. By the end of this guide, you will have the core competencies required to start building your own dynamic models.

About OpenModelica

OpenModelica is a free, open-source environment based on the Modelica modeling language. It is specifically designed for modeling, simulating, optimizing, and analyzing complex dynamic systems.

The software is actively developed by the **Open-Source Modelica Consortium (OSMC)**, a non-profit, non-governmental organization. The consortium is operated as a project of RISE SICS East AB in close collaboration with Linköping University.

OpenModelica is widely used in both academic and industrial environments. Industrial applications span a diverse range of sectors, including:

- **Power Plant Optimization:** Enhancing thermal efficiency and operational flexibility.
- **Automotive Engineering:** Simulating thermal management and fluid dynamics.
- **Water Treatment:** Modeling biochemical processes and hydraulic control systems.

The transparency of open-source software allows engineers and researchers to inspect, customize, and validate the underlying mathematical models as a critical advantage for those who demand complete clarity in their simulation workflows.

Prerequisites

This guide is designed for absolute beginners. There are no strict prerequisites for following along.

- **Hardware:** A standard modern computer capable of running Windows or Linux.
- **Attitude:** A willingness to learn and an eagerness to explore mathematics behind process engineering simulations.
- **Mindset:** No prior experience with Modelica or OpenModelica is required. We will build your knowledge from the ground up.

Acknowledgements

A huge thank you is owed to the global **OpenModelica community** and the broader **open-source ecosystem**. The developers, maintainers, and contributors have dedicated countless hours to creating a transparent, high-quality simulation environment that is freely accessible to all. Without

their passion for democratizing engineering software, this guide and the learning it facilitates would not be possible.

About the Author

Viraj Desai is an engineer who enjoys exploring open-source tools, learning new skills, and collaborating with the engineering community.

Disclaimer

Important Notice: Educational Use Only

All exercises, models, and mathematical representations presented in this manual are **strictly restricted to learning and educational purposes only**.

The author is currently in an active learning phase and is continuously exploring the capabilities of OpenModelica. The models contained within these pages are **simplified representations** of complex physical phenomena and are **not intended for direct application in real-world industrial environments**.

No responsibility is assumed by the author or the Open Source Modelica Consortium for any loss, damage, or operational failure resulting from the use of the information contained herein.

A Note to the Community: As this is a work-in-progress born from a learning journey, constructive feedback and cooperative insights are deeply appreciated. Let us grow this knowledge base together.

For real-world design, operational decisions, and safety-critical applications, always consult qualified professional engineers and utilize validated, commercially-supported software.

TABLE OF CONTENTS

1	What is OpenModelica?	6
1.1	The Short Definition	6
1.2	Why Should a Process Engineer Care?	6
1.3	The Ecosystem: How It Works	6
1.4	The Brain: The OpenModelica Compiler (OMC)	7
2	Installation of OpenModelica	7
2.1	Windows Installation	7
2.2	Linux Installation (Ubuntu/Debian)	8
2.3	macOS Installation	8
2.4	Verifying Your Installation	9
3	Navigating the OMEdit Interface	9
3.1	Launching OMEdit	9
3.2	The Main Window Layout	10
3.3	Customizing Your Workspace	12
4	The Building Blocks, Modelica Language Basics	13
4.1	Core Data Types	13
4.2	Structure of a Model	13
5	Creating and Writing a Model File	13
5.1	A Simple Code Block (The Time Function)	15
5.2	Understanding Packages and Modules	16
5.3	Running and Visualizing the Simple Code Block (A Preview)	16
6	Exporting Simulation Data, A Deep Dive Using <i>SimpleRamp</i>	19
6.1	Setting Up Our Test Case	19
6.2	Understanding the Simulation Output Files (Where is my data?)	20
6.3	Export Method 1: The Graphical Export (Plots)	20
6.4	Export Method 2: The CSV (Comma Separated Values) Export	21
7	References and Source Code	22

List of Figures

Figure 1 OMEdit Launch Screen	10
Figure 2 Menu Ribbon of OMEdit	10
Figure 3 Toolbar of OMEdit	10
Figure 4 Libraries of OMEdit	10
Figure 5 Extended View of Libraries of OMEdit	11
Figure 6 Diagram View	12
Figure 7 Message Logs	12
Figure 8 Canvas View of New Model	14
Figure 9 Text View of the New Model	15
Figure 10 Model Checks	17
Figure 11 Logs Checks	17
Figure 12 Simulate	17
Figure 13 Simulation Setup	18
Figure 14 Plots	19
Figure 15 Output File	20
Figure 16 Plots Display	21
Figure 17 Tool Ribbon to export results as CSV	21

List of Tables

No table of figures entries found.

1 WHAT IS OPENMODELICA?

Welcome to the world of digital process simulation. If you are a process engineer, you are likely familiar with the challenges of designing, optimizing, and troubleshooting complex systems whether they involve heat exchangers, distillation columns, or entire power plants. You need tools that can handle math without getting in the way of engineering.

This is where **OpenModelica** comes in.

1.1 THE SHORT DEFINITION

OpenModelica is a **free, open-source environment** designed for modeling, simulating, and optimizing complex dynamic systems. Think of it as a highly capable "virtual test lab" for your processes. Unlike some commercial software that feels like a "black box," OpenModelica is transparent and highly customizable, making it an excellent choice for engineers who want to understand the underlying physics and mathematics of their processes.

1.2 WHY SHOULD A PROCESS ENGINEER CARE?

While OpenModelica is a general-purpose tool, it is incredibly powerful for our specific field. Here is why you should pay attention:

1. **It Handles Physics:** Modelica (the language behind OpenModelica) is equation-based. This means you write equations that describe physical phenomena (like mass balances, energy balances, and phase equilibria) directly. The software figures out how to solve them.
2. **Real-World Applications:** It isn't just academic. OpenModelica is already being used in industrial settings you will recognize:
 - a. **Power Plant Optimization:** Engineers use it alongside other software to improve the efficiency of thermal power cycles.
 - b. **Automotive (Control Systems):** While not our primary focus, it shows the software's robustness in handling fluid and thermal dynamics.
 - c. **Water Treatment:** It is used to simulate and optimize the complex biochemical reactions and hydraulics in treatment plants.
3. **Cost and Collaboration:** As an open-source tool, there are no expensive license fees. This allows you to share models freely with colleagues, partners, or the academic community without worrying about proprietary software barriers.

1.3 THE ECOSYSTEM: HOW IT WORKS

OpenModelica isn't just a single program; it is a suite of tools designed to fit how you work. As a process engineer, you will likely interact with a few key components:

- a. **OMEdit (The Graphical Interface):** This is your primary "canvas." It is a user-friendly GUI where you can drag and drop components (like pumps, valves, and tanks) onto a diagram and connect them. It allows you to visualize your process flow diagram (PFD) while the software builds the mathematical model in the background.

- b. **OMShell (The Command Line):** Think of this as the "terminal" for the software. It allows you to run scripts, perform quick calculations, and start simulations without the GUI. It is useful for batch processing or running repetitive tasks.
- c. **OMPython (The Python Interface):** This is the "Pro Engineer" tool. If you are comfortable with Python, this allows you to call OpenModelica functions directly from your Python environment. You can combine process simulation with your favorite data analysis libraries (like Pandas or NumPy) or even connect it to machine learning algorithms for predictive maintenance.
- d. **OMNotebook (The Interactive Document):** This is a digital "engineering notebook." It allows you to mix text, equations, and simulation results in one document. It is particularly useful for documenting your design rationale and sharing "living" reports that can be updated by simply re-running the simulation.

1.4 THE BRAIN: THE OPENMODELICA COMPILER (OMC)

Under the hood, the **Compiler (OMC)** is the engine that translates your graphical process diagram into high-performance C code. This code is then linked to solvers to compute the numerical results. OMC ensures that your simulations are fast and reliable, even for large-scale industrial problems.

2 INSTALLATION OF OPENMODELICA

Welcome to the practical part of our journey. Before we can start modeling our first process, we need to get the software up and running on your machine. This chapter provides a straightforward, step-by-step guide to installing OpenModelica on Windows, Linux, and macOS.

The good news is that OpenModelica is **cross-platform** and installation is generally straightforward, regardless of your operating system.

2.1 WINDOWS INSTALLATION

For most Windows users, installation is as simple as downloading and running an installer.

1. Download the Installer:
 - a. Navigate to the official OpenModelica download page. The most straightforward way is to visit the OpenModelica website and find the download section for Windows.
 - b. You will typically find a direct link to the latest **.exe** installer file. The file is large (**around 1.85 GB**), so ensure you have a stable internet connection and sufficient time for the download.
2. Run the Installer:
 - a. Locate the downloaded file (e.g., **OpenModelica-v1.27.0-64bit.exe**) and double-click it to run.
 - b. Follow the on-screen instructions provided by the installation wizard.

3. Important Precaution for Windows Users:
 - a. OpenModelica may not handle non-English (double-byte) characters well.
 - b. This means your **Windows user account name** must contain **only standard English letters and numbers**.
 - c. If your current username contains special characters, you should create a new, simple user account (e.g., "ProcessEng") and log in with it **before** installing OpenModelica.
 - d. The installation path should also avoid spaces or special characters.

2.2 LINUX INSTALLATION (UBUNTU/DEBIAN)

For users on Debian or Ubuntu-based systems, OpenModelica is available through a dedicated repository. This method uses the *apt* package manager, making installation and updates very easy. We'll be installing the *openmodelica* package, which includes the main compiler and the OMEdit GUI.

Note: For a smaller installation without the graphical interface, you can use *sudo apt install --no-install-recommends omc*.

1. Install Prerequisites:
 - a. First, open a terminal and install the necessary tools to add a new repository:
`sudo apt-get update`
`sudo apt-get install ca-certificates curl gnupg lsb-release`
2. Add the OpenModelica GPG Key:
 - a. This step ensures your system trusts the software from the OpenModelica repository.
`curl -fsSL https://build.openmodelica.org/apt/openmodelica.asc | sudo gpg --dearmor -o /usr/share/keyrings/openmodelica-keyring.gpg`
3. Add the OpenModelica Repository:
 - a. The following command automatically detects your Linux distribution and adds the correct repository. It uses the **stable** release branch, which is recommended for productive usage.
`echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/openmodelica-keyring.gpg] https://build.openmodelica.org/apt $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/openmodelica.list`
4. Update and Install:
 - a. Finally, update your package list and install OpenModelica.
`sudo apt-get update`
`sudo apt-get install openmodelica`

2.3 MACOS INSTALLATION

The official OpenModelica project no longer provides pre-built installers for macOS. For macOS users, the recommended approach is to use a virtual machine or the Windows Subsystem for Linux

(WSL) on a Windows machine. If you are using a Mac and need to run OpenModelica, the most common workaround is to run it within a Linux environment. This can be achieved using:

- Docker: You can run a Linux container with OpenModelica pre-installed.
- A Virtual Machine: Using software like VirtualBox or VMware to run a Linux distribution (like Ubuntu) and then install OpenModelica within that virtual machine.

2.4 VERIFYING YOUR INSTALLATION

Once the installation is complete, it's good practice to verify that everything is working correctly. Open a **Command Prompt (Windows)** or a **Terminal (Linux/macOS)** and type the following command:

```
omc -- version
```

If the installation was successful, this command will print the version number of the OpenModelica Compiler (OMC) that is now installed on your system. Seeing this version number confirms that the core of OpenModelica is ready to use.

3 NAVIGATING THE OMEDIT INTERFACE

Before we write a single line of code or connect a single pipe, it is essential to get comfortable with the "cockpit" of OpenModelica: the OMEdit graphical user interface. As a process engineer, you will spend a significant amount of time here, whether you are browsing for components, debugging a model, or adjusting parameters.

This chapter is a guided tour of the OMEdit workspace. We will identify the key panels, ribbons, and menus so you know exactly where to look when you need to perform a specific action.

3.1 LAUNCHING OMEDIT

Locate the OpenModelica folder in your Start Menu (Windows) or Applications (Linux) and click the **OMEdit** icon. Once the splash screen disappears, you will be greeted by the main window.

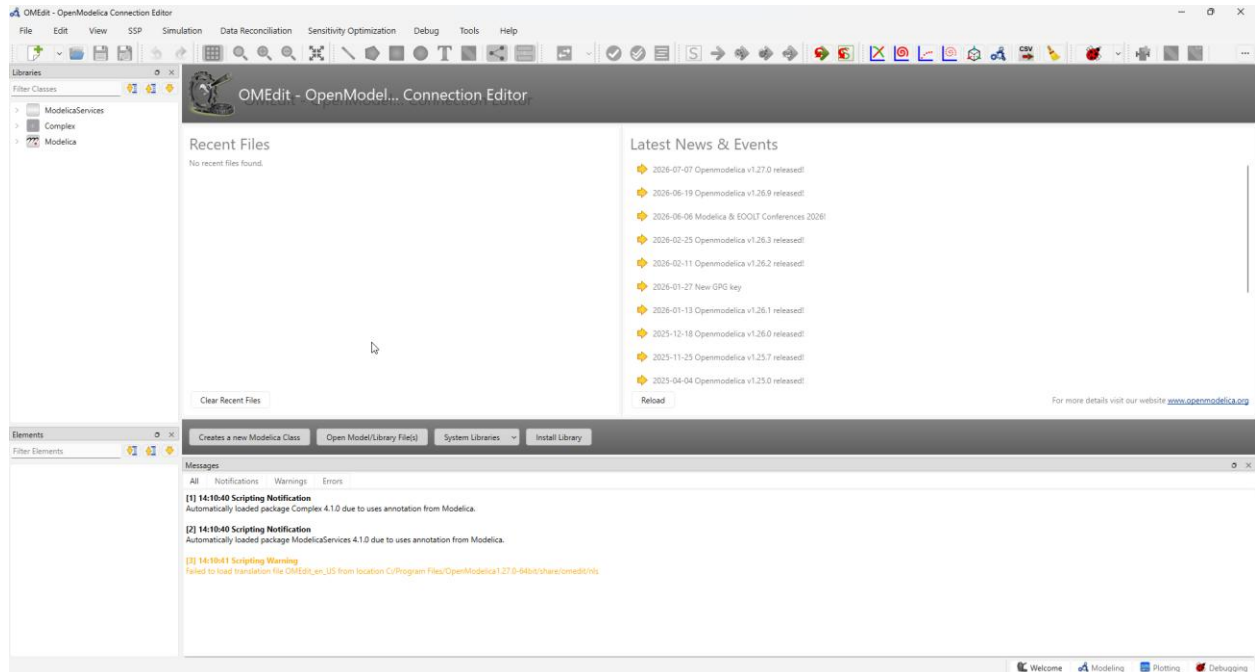


Figure 1 OMEdit Launch Screen

3.2 THE MAIN WINDOW LAYOUT

The OMEdit interface is divided into several distinct, resizable areas. Think of it as your engineering workstation:

- The Menu Bar & Ribbon (Top): This is the command center. It contains drop-down menus (File, Edit, View, Simulation, Tools, Help) and a customizable toolbar directly below it.

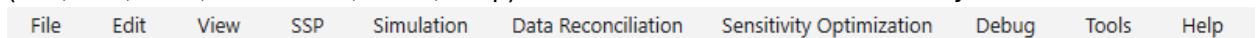


Figure 2 Menu Ribbon of OMEdit

- Key items to note: The **"New Model"** icon, the **"Save"** button, and the critical **"Simulate"** (S button) and **"Check"** (tick mark) buttons.

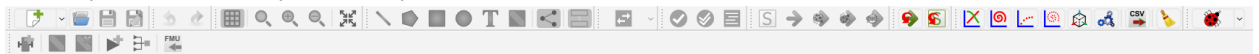


Figure 3 Toolbar of OMEdit

- **The Library Browser (Left Panel):** This is your "parts catalog". It lists all the loaded component libraries.

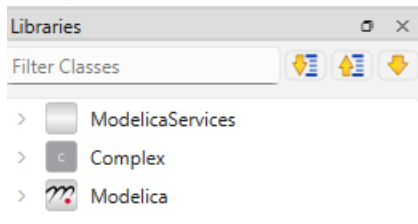


Figure 4 Libraries of OMEdit

- The most important entry here is the **Modelica Standard Library (MSL)**. You can expand the tree structure (e.g., *Modelica -> Fluid -> Vessels*) to find pre-built components.

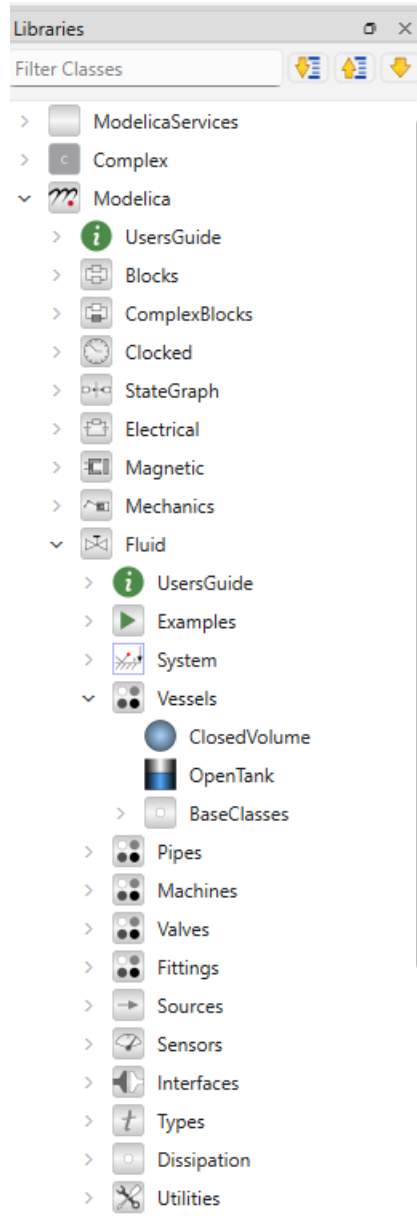


Figure 5 Extended View of Libraries of OMEdit

- You can also right-click here to load additional custom libraries or package files (.mo files) that you might receive from colleagues.
- The Modeling Canvas (Center):** This is your primary workspace.
- It features two main views, toggleable via tabs at the bottom of the canvas: **Diagram View** (the visual, icon-based P&ID-like layout) and **Text View** (the underlying Modelica source code).
- You can zoom in and out using *Ctrl + Mouse Wheel* and pan by holding down the *CTRL* key while dragging.

- **Modeling Perspective:** Shows the Library Browser and the Component Browser (listing variables inside the selected component).

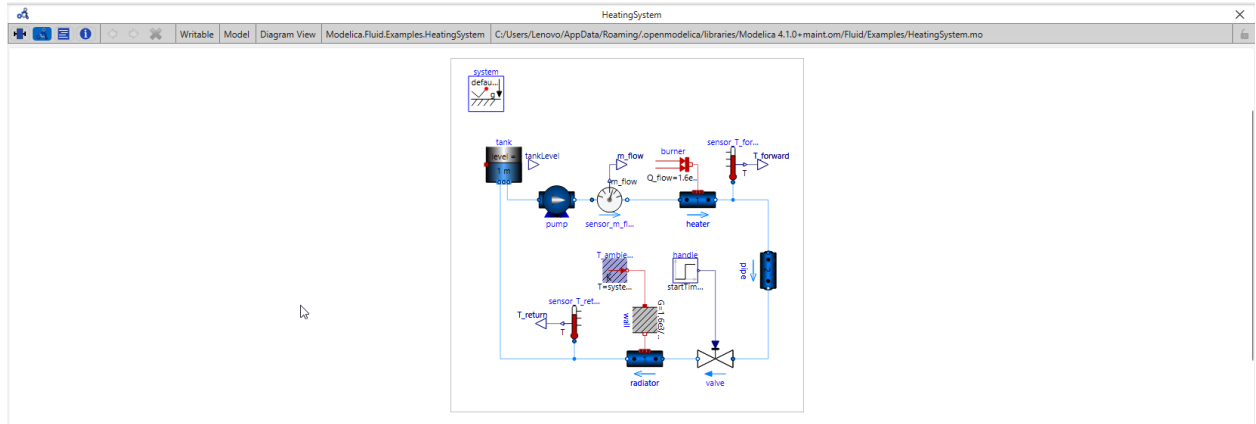


Figure 6 Diagram View

- **Simulation Perspective:** Shows the variable browser, plot manager, and simulation output.
- You can switch between these to keep your workspace uncluttered.
- **The Message Log (Bottom Panel):** This is your diagnostic terminal.

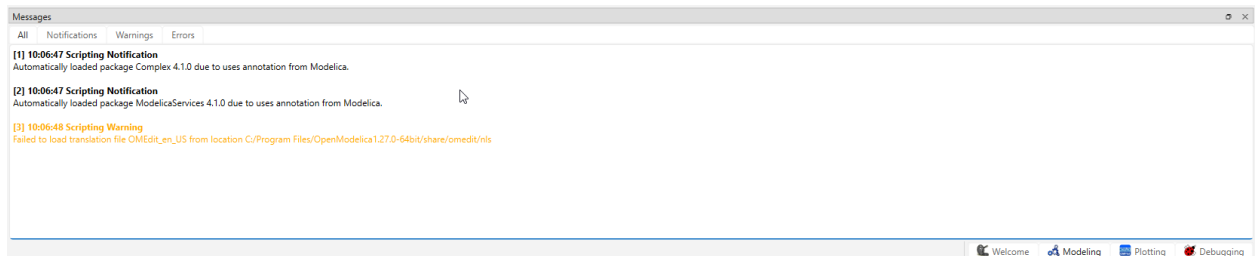


Figure 7 Message Logs

- Whenever you check, simulate, or translate a model, the output appears here.
- **Crucial for engineers:** If your model is "red" (has errors), this panel will tell you exactly which line of code or which connection is causing the issue. Look for lines highlighted in red text.
- **The Status Bar (Very Bottom):** Displays quick information like the current working directory, the selected solver, and simulation status (Idle, Translating, Simulating).

3.3 CUSTOMIZING YOUR WORKSPACE

You are not stuck with the default layout. If the message log is too small, you can drag its top border upward. If you prefer the Library Browser on the right side, you can drag and drop its tab to a new docking location.

Now that we know where everything is, we can start understanding the language that makes this all work.

4 THE BUILDING BLOCKS, MODELICA LANGUAGE BASICS

While OMEdit allows you to build models graphically, every diagram is backed by powerful, text-based code. Understanding the fundamentals of the Modelica language is like learning to read the P&ID legend; it allows you to troubleshoot errors, write custom logic, and truly understand what the software is doing.

In this chapter, we will cover the essential data types, code structure, and how OpenModelica organizes libraries and packages.

4.1 CORE DATA TYPES

Like any programming or scripting language, Modelica uses specific types to define data:

Real: Used for floating-point numbers (engineering quantities with decimals). Example: *3.14*, *299792458.0*, or *5.0*.

Integer: Used for whole numbers, usually for counters or discrete indices. Example: *1*, *10*, *-5*.

Boolean: Represents true/false states, often used for on/off switches or conditional logic. Example: *true*, *false*.

String: Used for textual information, such as component names or file paths. Example: *Pump_101*.

4.2 STRUCTURE OF A MODEL

A basic Modelica model is contained within a *model ... end* block. Inside, the code is typically organized into three distinct sections:

1. **Variable Declarations:** Where you define the quantities involved. You can assign attributes here.
 - a. **Real pressure;** (a variable that changes during simulation)
 - b. **parameter Real density = 1000;** (a constant parameter you set before simulation)
 - c. **constant Real pi = 3.14159;** (a true constant)
2. **Equations (equation section):** This is the heart of Modelica. You write **declarative** mathematical relationships. The compiler figures out how to solve them. (e.g., ***pressure = density * 9.81 * height;***).
3. **Algorithms (algorithm section):** If you need procedural logic (like *if-else* statements or *for* loops that execute step-by-step), you place them here.

5 CREATING AND WRITING A MODEL FILE

Before we can write any code, we need to create a new file to hold our work. OpenModelica gives you a few ways to do this. Let us start with the most straightforward method inside OMEdit.

Method A: Creating a New Model via OMEdit

1. In the top menu bar, navigate to File → New → Modelica Class.

2. A dialog box will pop up. Fill it in as follows:
 - a. Name: SimpleRamp (no spaces, use CamelCase).
 - b. Class Type: Select Model (this is your standard simulation container).
 - c. Extends: Leave this blank.
 - d. Insert: Leave this blank.
3. Click OK. You will now see a blank, rectangular canvas (the Diagram View) representing your new model.

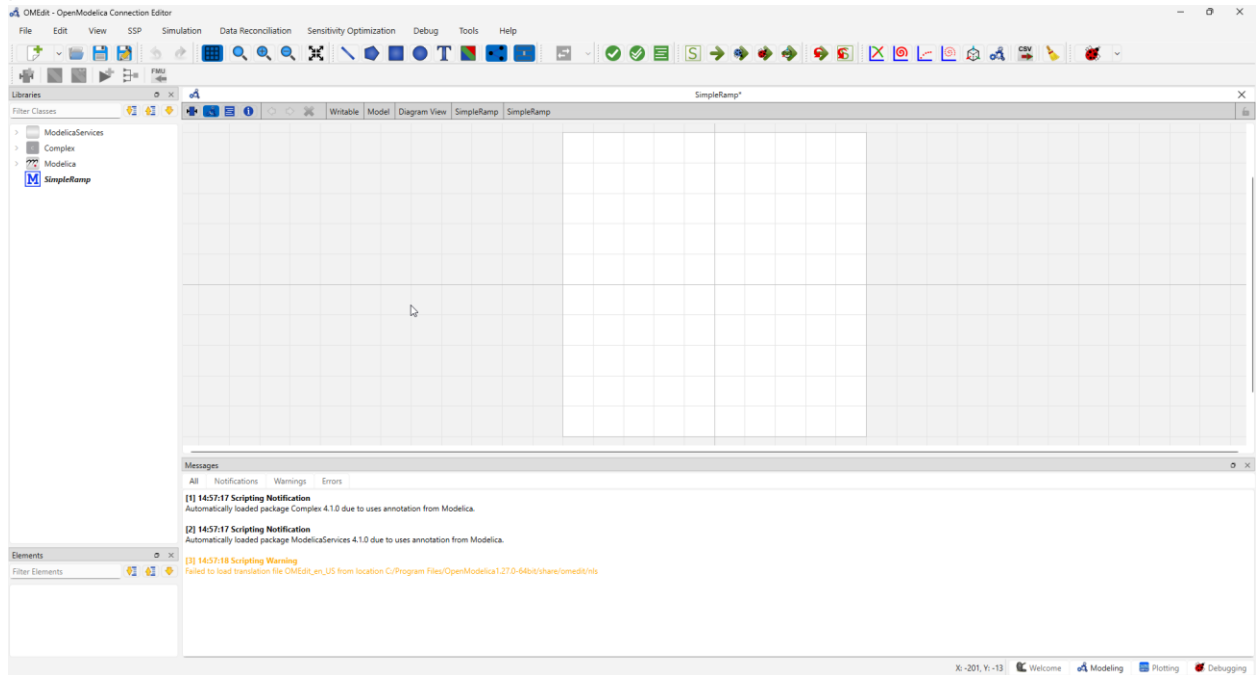


Figure 8 Canvas View of New Model

4. Switching to Text View: At the bottom of the canvas area, click the tab labeled Text View. This is where we will write our raw Modelica code. You will notice OMEdit has already generated the empty skeleton for you:
`model SimpleRamp`
`equation`

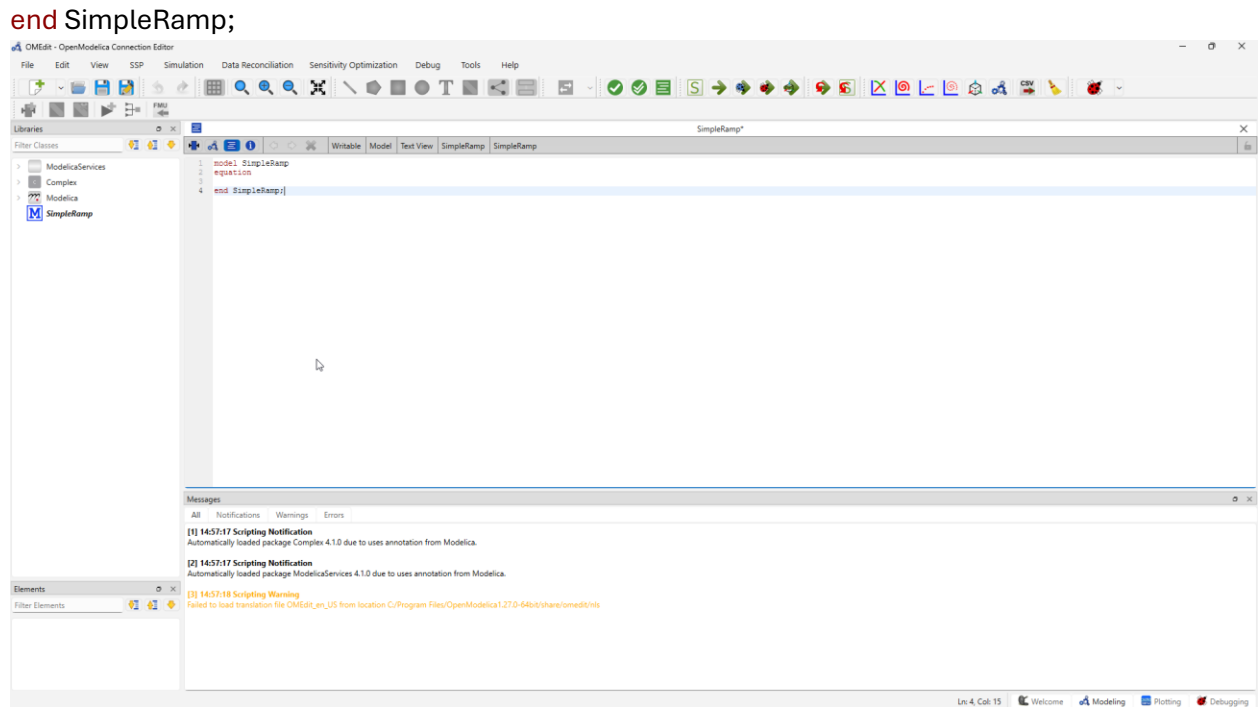


Figure 9 Text View of the New Model

Method B: Creating a Pure Text File (External Editor)

If you prefer using an external editor like Notepad++ or VS Code (which offer syntax highlighting), you can:

1. Open your favorite text editor.
2. Type your Modelica code from scratch.
3. Save the file with a .mo extension (e.g., SimpleRamp.mo).
4. In OMEdit, go to File → Import → Modelica File(s) and select your .mo file. It will appear in your Library Browser.

Pro Tip for Engineers: Always keep your working directory organized. Create a dedicated folder (e.g., C:\OpenModelica_Workspace) and save all your models there. OMEdit defaults to this folder for file operations

5.1 A SIMPLE CODE BLOCK (THE TIME FUNCTION)

Now that we have our SimpleRamp model open in the Text View, let us populate it with our first piece of custom logic. We are going to write a model that doesn't simulate a physical process, but rather a simple mathematical function of time. This is excellent for understanding how the simulation engine handles dynamic variables.

Type or paste the following code into the Text View, replacing the empty skeleton:

```
model SimpleRamp
  Real output_value; // A variable to hold our result
```

```
parameter Real slope = 2.0; // A user-defined parameter
equation
  output_value = slope * time; // 'time' is a built-in simulation variable
end SimpleRamp;
```

Let us break down what is happening here:

- *Real output_value*; declares a variable that will change continuously during the simulation.
- *parameter Real slope = 2.0*; declares a constant that we can tweak before the simulation starts.
- *equation* starts the mathematical section.
- *output_value = slope * time*; uses *time*, which is a built-in global variable in Modelica that represents the simulation clock (starting at 0).

If we simulate this for 10 seconds, *output_value* will start at 0 and linearly increase to 20. No iterative solvers are stressed here; it is a direct calculation.

5.2 UNDERSTANDING PACKAGES AND MODULES

How does OpenModelica keep track of thousands of components like pumps, valves, and controllers? Through Packages.

- A Package is simply a container (like a folder on your computer) that groups related models, functions, and constants together.
- For example, the *Modelica.Fluid* package contains sub-packages for *Sources*, *Vessels*, and *Pipes*.
- To reference a component, you use dot-notation. For example, *Modelica.Blocks.Math.Sin* refers to the Sine function block located inside the Math sub-package.
- How files map to packages: When you save a model named *MyPackage.MyComponent*, OpenModelica expects it to be located in a folder structure that matches this hierarchy (e.g., *MyPackage/MyComponent.mo*).
- Loading external libraries: If a colleague sends you a custom .mo library, you can load it by right-clicking in the Library Browser and selecting Load File or Load Directory. You do not need to place it in a specific system folder; OMC resolves the paths automatically.

Pro Tip: Creating your own packages (File -> New -> Package) is best practice for standardizing components across your engineering team. It allows you to reuse your custom valve or reactor models across multiple projects without copying and pasting code.

5.3 RUNNING AND VISUALIZING THE SIMPLE CODE BLOCK (A PREVIEW)

Now that our SimpleRamp model is written, let us run a quick check and simulation to prove our code works. We will keep this brief, as we will dive much deeper into the simulation setup, initialization, and plotting tools in the next chapter.

1. Check the Model: Click the Check button (the blue tick mark on the toolbar).



Figure 10 Model Checks

2. Look at the Message Log at the bottom. You should see a message in blue: *Class 'SimpleRamp' has been successfully checked.* This tells us our equations are balanced (1 unknown, 1 equation).

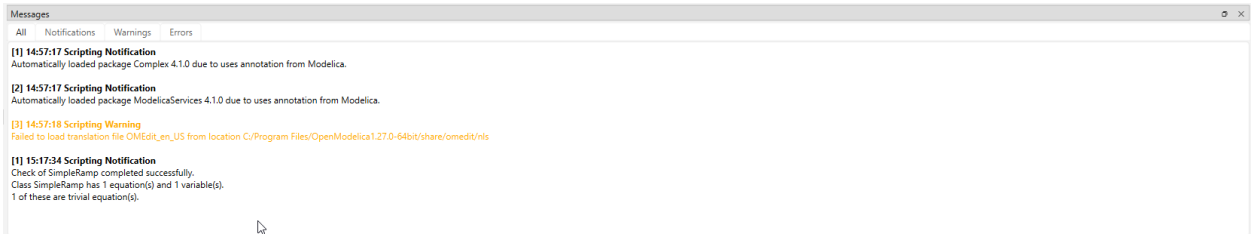


Figure 11 Logs Checks

3. **Simulate:** Click the Simulate button (the S icon) on the toolbar.



Figure 12 Simulate

4. **Set Time:** In the dialog that appears, set Stop Time to 10.0 seconds. Leave everything else as default and click Simulate.

OMEdit - Simulation Setup - SimpleRamp

Simulation Setup - SimpleRamp

General Interactive Simulation Translation Flags Simulation Flags Output Linearize

Simulation Interval

Start Time: 0 s

Stop Time: 1 s

☒ Number of Intervals: 500 ^ v

☐ Interval: 0.002 s

Integration

Method: dassl ^

Tolerance: 1e-06

Jacobian: ^

Options

☒ Root Finding

☒ Restart After Event

Initial Step Size: ^

Maximum Step Size: ^

Maximum Integration Order: 5 ^ v

C/C++ Compiler Flags (Optional): ^

Number of Processors: 12 ^ v Use 1 processor if you encounter problems during compilation.

☐ Build Only ☐ Launch Transformational Debugger

☐ Launch Algorithmic Debugger ☐ Launch Animation

☐ Save experiment annotation inside model i.e., experiment annotation

☐ Save translation flags inside model i.e., __OpenModelica_commandLineOptions annotation

☐ Save simulation flags inside model i.e., __OpenModelica_simulationFlags annotation

☒ Simulate

OK Cancel

Figure 13 Simulation Setup

5. **View Plot:** Once the simulation finishes, the Plot window will open. Look for output_value in the variable list on the left, click it, and you will see a perfectly straight diagonal line

climbing from 0 to 20 over the 10 seconds.

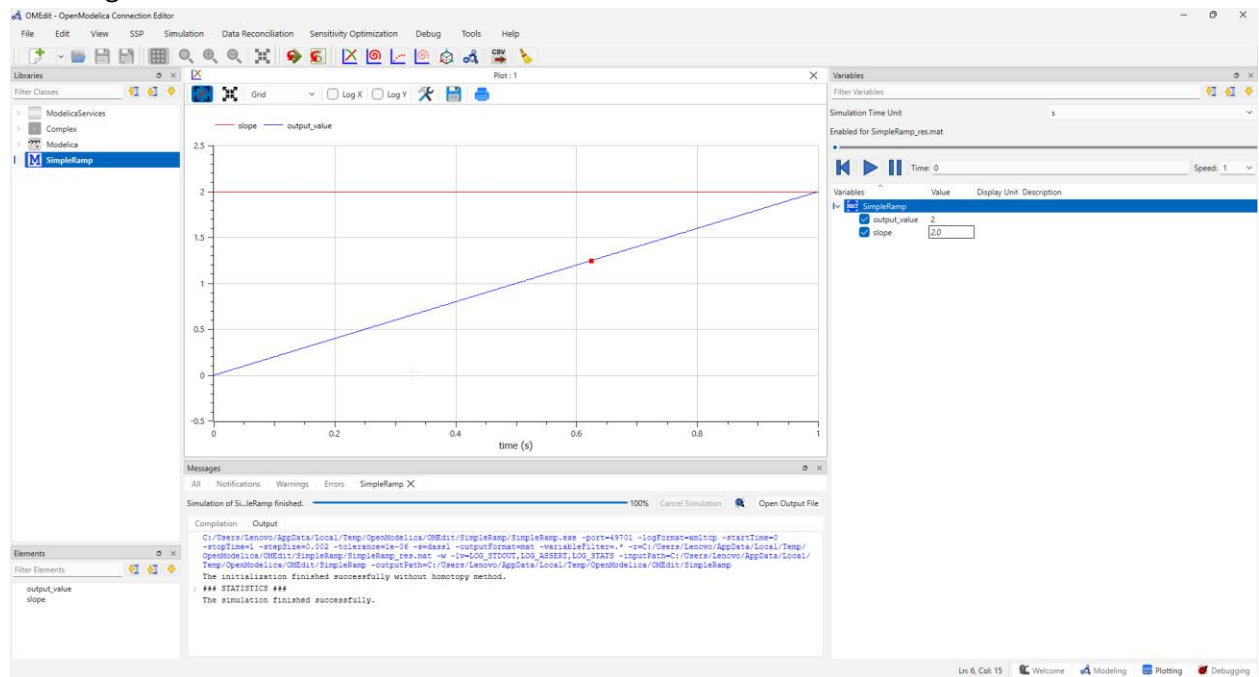


Figure 14 Plots

6 EXPORTING SIMULATION DATA, A DEEP DIVE USING *SIMPLERAMP*

In Chapter 5, we briefly touched upon exporting a plot image and exporting a CSV file. Now that we have a solid grasp of the simulation workflow, let us zoom in specifically on **data management**.

As an engineer, the simulation plots inside OMEdit are great for visual inspection, but you will ultimately need the raw numerical data to create publication-quality charts, perform statistical analysis in Python, or generate compliance reports in Excel.

In this chapter, we will use our simple *SimpleRamp* model (which we coded in Chapter 4) as a clean, predictable test case to explore every data export option OpenModelica offers. By the end, you will know exactly how to extract your results and where to find them on your hard drive.

6.1 SETTING UP OUR TEST CASE

Before we export, let us ensure our model is ready to go.

1. Open OMEdit and ensure the *SimpleRamp* model is loaded in your **Library Browser**.
2. Double-click it to open it in the **Text View** to confirm the code is present:

```
model SimpleRamp
```

```
  Real output_value; // A variable to hold our result
```

```
  parameter Real slope = 2.0; // A user-defined parameter
```

```
equation
```

```
  output_value = slope * time; // 'time' is a built-in simulation variable
```

end SimpleRamp;

- Click the **Simulate** button. Keep the default settings (Stop Time = 1.0 second for now but change it to **10.0** seconds for a better curve). Click **Simulate**.
- Wait for the "**Simulation done**" message. Now, the raw results exist somewhere on your computer. Let us go find them.

6.2 UNDERSTANDING THE SIMULATION OUTPUT FILES (WHERE IS MY DATA?)

When OpenModelica simulates a model, it generates several files in your **Working Directory**. To find this directory:

- Look at the very bottom **Status Bar** of OMEdit. You will see a path (e.g., *C:/Users/YourName/OpenModelica_Workspace*).
- Open your File Explorer and navigate it to this folder. Or you may simply click on the Open Output File

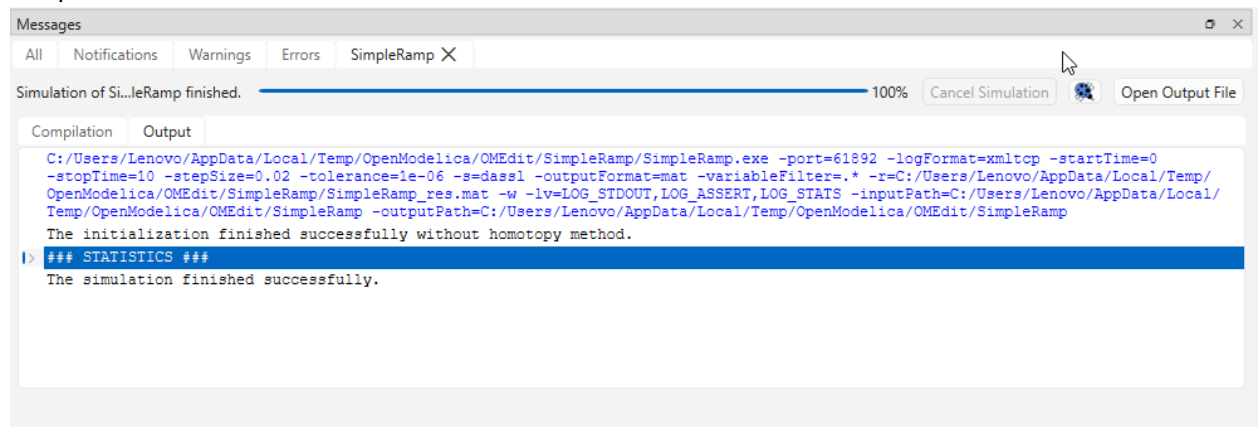


Figure 15 Output File

Pro Tip: OpenModelica defaults to the *.mat* format because it is fast to write and universally readable by scientific software.

6.3 EXPORT METHOD 1: THE GRAPHICAL EXPORT (PLOTS)

This is the quickest method if you just need a visual snapshot.

- When the simulation finishes, the **Plotting Perspective** opens automatically.

2. Select the *output_value* variable on the left to display the incline line.

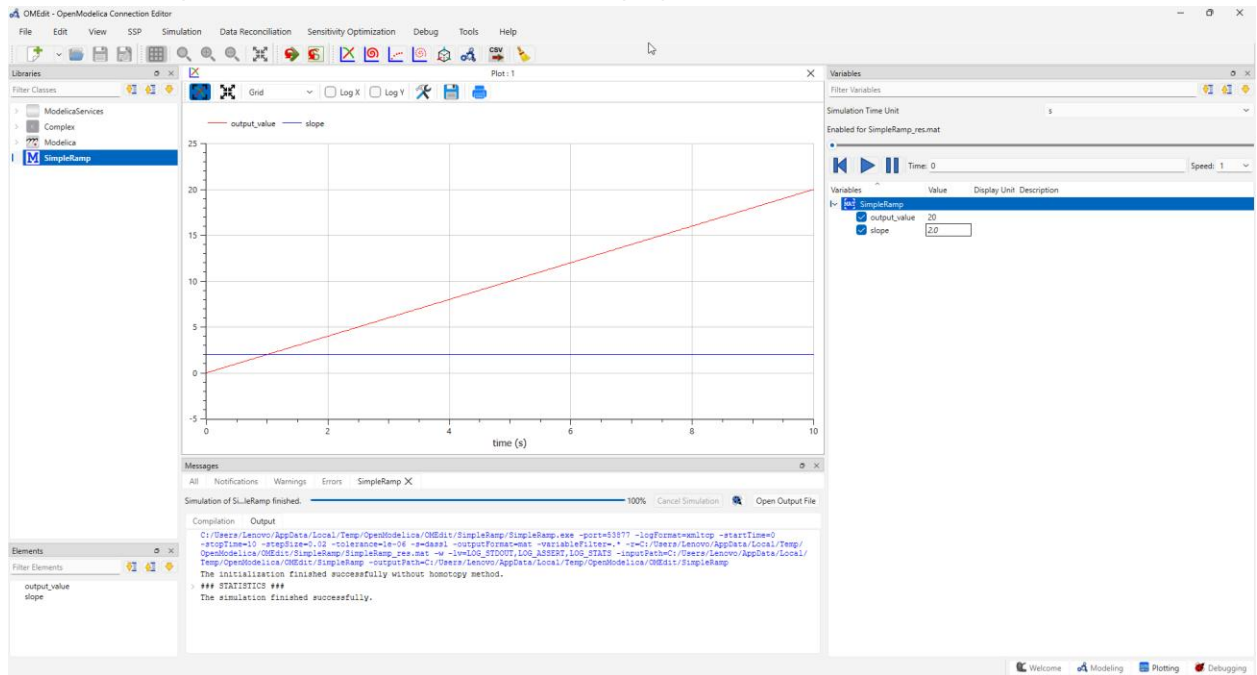


Figure 16 Plots Display

3. Look for the **Export** icon in the plot window's toolbar (it looks like a small floppy disk or an image icon, often with a green arrow).
4. Click it. A dialog will ask you for:
5. **File Name:** e.g., *Ramp_Result*.
6. **File Type:** Choose between *.png* (for quick pasting into Word), *.svg* (scalable vector graphic for presentations).
7. Click **Save**. The image is saved directly to your working directory.

6.4 EXPORT METHOD 2: THE CSV (COMMA SEPARATED VALUES) EXPORT

If you want to open your data in Microsoft Excel or import it into a Pandas DataFrame in Python, CSV is the gold standard.

1. With the *SimpleRamp* model still active, go to the top menu bar and click **Simulation** → **Export** → **CSV**.



Figure 17 Tool Ribbon to export results as CSV

2. A file dialog will pop up. By default, it suggests saving *SimpleRamp_res.csv* in your working directory. Click **Save**.
3. Navigate to your working directory and open this *.csv* file with a text editor or Excel. You will see columns for *time* and *output_value* perfectly aligned.
4. **Excel Users:** If the data looks like one long string, use Excel's **Data** → **From Text/CSV** import feature and specify the delimiter as a comma.

Now that you have mastered extracting data from OpenModelica, you have the tools to take your simulation results out of the software and into your engineering decision-making process. In the next section (or the next guidebook), we will move beyond the *SimpleRamp* and apply these skills to actual industrial equipment

7 REFERENCES AND SOURCE CODE

1. [OpenModelica User Guide](#)
2. [Source Code for SimpleRamp](#)